



Exp :10 Write the python program to implement A* algorithm

Aim:

To write a Python program to implement the A* (A-Star) search algorithm to find the shortest path from a start node to a goal node.

Algorithm:

Step 1: Define a function for the A* algorithm that accepts the graph, heuristic values, start node, and goal node.

Step 2: Initialize an open list (containing nodes to be evaluated) and put the start node in it.

Step 3: Initialize a closed list (containing nodes already evaluated) as empty.

Step 4: Create a dictionary g to store the actual distance from the start node to each node, setting the start node's distance to 0.

Step 5: Create a dictionary parents to keep track of the path, setting the start node's parent to itself.

Step 6: While the open list is not empty, find the node n with the lowest total cost $f(n) = g(n) + h(n)$ (actual cost + heuristic cost).

Step 7: If node n is equal to the goal node or if node n has no neighbors, stop searching.

Step 8: For each neighbor m of node n, calculate its new g cost.

Step 9: If neighbor m is not in the open or closed list, add it to the open list, set its parent to n, and record its g cost.

Step 10: If neighbor m is already in the open or closed list but the new g cost is lower, update its g cost, change its parent to n, and move it back to the open list if it was in the closed list.

Step 11: Remove node n from the open list and add it to the closed list.

Step 12: Once the goal node is reached, reconstruct the path by tracing backward using the parents dictionary from the goal node to the start node, then print the path.

Program:

```
def astar_search(graph, heuristics, start, goal):  
    open_list = set([start])  
    closed_list = set()  
    g = {}
```

```
parents = {}
```

```
g[start] = 0
```

```
parents[start] = start
```

```
while len(open_list) > 0:
```

```
    n = None
```

```
    for v in open_list:
```

```
        if n is None or g[v] + heuristics[v] < g[n] + heuristics[n]:
```

```
            n = v
```

```
    if n == goal or graph[n] is None:
```

```
        pass
```

```
    else:
```

```
        for (m, weight) in graph[n]:
```

```
            if m not in open_list and m not in closed_list:
```

```
                open_list.add(m)
```

```
                parents[m] = n
```

```
                g[m] = g[n] + weight
```

```
            else:
```

```
                if g[m] > g[n] + weight:
```

```
                    g[m] = g[n] + weight
```

```
                    parents[m] = n
```

```
                if m in closed_list:
```

```
                    closed_list.remove(m)
```

```
                    open_list.add(m)
```

```
    if n == goal:
```

```
path = []  
while parents[n] != n:  
    path.append(n)  
    n = parents[n]  
path.append(start)  
path.reverse()  
print("Shortest Path found:", path)  
return path
```

```
open_list.remove(n)
```

```
closed_list.add(n)
```

```
print("Path does not exist!")
```

```
return None
```

```
graph = {  
    'A': [('B', 2), ('E', 3)],  
    'B': [('C', 1), ('G', 9)],  
    'C': [],  
    'E': [('D', 6)],  
    'D': [('G', 1)],  
    'G': []  
}
```

```
heuristics = {  
    'A': 11,  
    'B': 6,  
    'C': 99,
```

```
'E': 7,  
'D': 1,  
'G': 0  
}
```

```
astar_search(graph, heuristics, 'A', 'G')
```

OUTPUT:

```
Shortest Path found: ['A', 'E', 'D', 'G']
```

Result: Thus, the program was executed and found that the output was found to be correct.